DAY3 - Image Management & Security:

Docker image tagging, storage, and reuse
Google Artifact Registry
Image vulnerability scanning (Trivy, GCP tools)
Secure image distribution and IAM roles
Hands-On Push images to Artifact Registry, run security scans

* What is Image Tagging?:

Tagging = giving a name + version to a Docker image

Format:

<image_name>:<tag>

Example:

myapp:1.0

** Tagging Strategy (Industry Standard) *

Tag	Meaning
myapp:1.0	Stable production version
myapp:latest	Default (latest build)
myapp:dev	Development version
myapp:1.0-rc	Release candidate

** Common Tagging Stages **

Tag	Meaning
dev	Development version
test	Testing version
rc	Release Candidate
latest	Latest stable
1.0	Final production version

** Typical Workflow **

myapp:dev → developers test
myapp:test → QA tagging
myapp:1.0-rc → almost final
myapp:1.0 → production release

* How to Build with RC Tag

docker build -t myapp:1.0-rc .

* How to Run

docker run -d -p 5000:5000 myapp:1.0-rc

* Why Use RC Tag?
Helps in: Final validation before release, UAT (User Acceptance Testing),Bug fixing without affecting production

Release Candidate is the final testing version before production release

Best Practices:

1. Never Rely Only on latest

latest is not truly "latest", it just means:

"last pushed image without a specific tag"

Problem:
- Unpredictable deployments
- Different environments may run different versions
- Hard to debug issues

2. Use Versioning (Semantic Versioning)
Follow standard:

MAJOR.MINOR.PATCH

Example: consider myapp:1.0.0

Version	Meaning
1.0.0	Initial release
1.1.0	New feature
1.1.1	Bug fix
2.0.0	Breaking change

Maintain Rollback Versions:
-> Always keep older images available

Why?

If something breaks in production:

docker run myapp:1.1.1 #Instant rollback

Windows 7 -> Stable version

Windows 8 --> Not stable -> The "Experimental" Fail

Windows 8.1 -> Limited to TABLET OS version

Windows 9 -> USER Skipped

Windows 10 -> Stable Version

Windows 10 -> Stable Version

Dangling images (unused layers)

```
jaykumar@Tiffany-instance-20250316-093931:~ docker images
REPOSITORY   TAG       IMAGE ID       CREATED          SIZE
<none>       <none>    5f0a1ad7b8f8   2 hours ago      1.19GB
<none>       <none>    5f0a1ad7b8f8   2 hours ago      1.19GB
<none>       <none>    8c0f4f7a6c0f   20 hours ago     1.19GB
<none>       <none>    8c0f4f7a6c0f   20 hours ago     1.19GB
mon-app      latest    5f0f6139a2b6   20 hours ago     1.19GB
<none>       <none>    5f0f6139a2b6   20 hours ago     1.19GB
myjrjs       latest    9517bed3d97    7 days ago       10.6MB
<none>       <none>    9517bed3d97    7 days ago       10.6MB
<none>       <none>    5c8b4e7c7f1    8 months ago     1.19GB
hello-world  latest    9c7a54a9a43c   11 months ago    13.3kB
busy         latest    b5bb9d8c3b7f   11 months ago    4.26MB
busy         v1        b5bb9d8c3b7f   11 months ago    4.26MB
jaykumar@Tiffany-instance-20250316-093931:~$
```

GCP - Artifact Registry:
Google Cloud Artifact Registry is the evolution of the now-deprecated Container Registry (GCR). It acts as a single, central location for managing both container images and language packages like Maven/npm and Python.

Key Features:

Multi-Format Support: Beyond just Docker/OCI images, it supports:

Language Packages: Java (Maven/Gradle), Node.js (npm), Python (PyPI), Go.

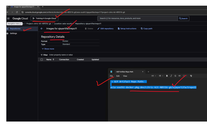
OS Packages: Debian (Apt) and RPM (Yum/DNF).

Generic: Versioned, immutable artifacts of any type.

Granular Access Control: Unlike the old Container Registry (which used Cloud Storage bucket permissions), Artifact Registry uses IAM at the repository level. You can give different permissions to different teams for specific repositories.

Regionality: You can host repositories in specific regions (e.g., asia-south1 for Mumbai/Bengaluru) to reduce latency and data transfer costs, or comply with data residency requirements.

Vulnerability Scanning: Automatically scans images for OS and vulnerabilities as they are uploaded.



Artifact Repository Path:

asia-south1-docker.pkg.dev/citric-kit-489316-g6/jayartifactrepo11

Image Naming Format:*

<region>-docker.pkg.dev/<project-id>/<repo-name>/<image>:<tag>

asia-south1-docker.pkg.dev/citric-kit-489316-g6/jayartifactrepo11/myapp:1.0

Step 1: Docker installation in GCP cloud Shell

Step 2: Authenticate Docker with GCP

** PUSH IMAGE FROM VM + ARTIFACT REGISTRY

Step 1: Set Project

gcloud config set project citric-kit-489316-g6

Step 2: Authenticate Docker

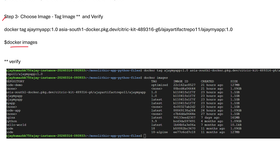
gcloud auth configure-docker asia-south1-docker.pkg.dev

Step 3: Choose Image - Tag Image ** and Verify

docker tag sqsmyapp:1.0 asia-south1-docker.pkg.dev/citric-kit-489316-g6/jayartifactrepo11/jajaymyapp:1.0

$docker images

** verify



Step 4: Push Image

*** Image vulnerability scanning (Trivy, GCP tools) **

Docker images can contain outdated packages, misconfigurations, or known security vulnerabilities. Scanning ensures your images are safe before deploying.

** Trivy (A simple and popular scanner)

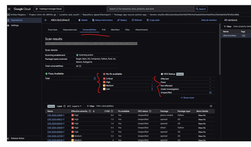
** Installation :

** Use standalone binary (easiest)

** Trivy - Image Scanning tool:

It is important to scan the image before pushing into production

trivy image monolithic-python-app:1.0

```
jaykumar@ip-172-31-0-100:~/monolithic-app-python-files$
jaykumar@ip-172-31-0-100:~/monolithic-app-python-files$ trivy image monolithic-python-app:1.0
```

Google Artifact Registry - Vulnerabilities -scanning :